COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	9/16/26	**DATE SUBMITTED:**	9/16/26
NAME:	Mike Justin A. Cabangon	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 **Sorting Algorithms**

1. PROCEDURE OUTPUT

CODE

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingalgo.h"

//bubble sort
// generate 100 random values
// create a function that will display the random generated values

const int maxSize = 100;
void displayArr(int arr[]);

int main() {
    // create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    // use time(0) as seed for random values
    std::srand(std::time(0));

    // populate the array with random values
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = std::rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArr(unsortedArr);
    return 0;
}
void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

OUTPUT

```
Unsorted Array:
22 53 60 92 54 4 32 21 75 60 97 51 25 85 76 73 37 66 81 2 26 26 26 57 2
82 9 29 6 33 51 15 44 5 73 29 47 28 13 9 32 97 44 95 24 60 39 67 58 75 18
74 59 91 61 35 95 11 89 13 38 67 9 51 54 83 25 95 18 93 14 62 71 67 68 88
47 94 88 83 20 48 27 42 85 53 91 73 58 33 42 50 56 27 60 85 60 20 40 11
```

OBSERVATION

The program generates 100 random integer values ranging from 0 to 99 using std::rand() % 100. The elements in the array are printed in unsorted order.

Table 7-1. Array of Values for Sort Algorithm Testing

CODE:Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingalgo.h"

//bubble sort
// generate 100 random values
// create a function that will display the random generated values

const int maxSize = 100;
void displayArr(int arr[]);

int main() {
    // create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    // use time(0) as seed for random values
    std::srand(std::time(0));

    // populate the array with random values
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = std::rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArr(unsortedArr);

    std::cout << "\nBubble Sort:\n";
    bubbleSort(unsortedArr, maxSize);
    displayArr(unsortedArr);
    return 0;
}
void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

CODE:Header

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>

// Bubble Sort
template <typename T>
void bubbleSort(T arr[], int arrSize) {
    for (int i = 0; i < arrSize - 1; i++) {
        for (int j = 0; j < arrSize - i - 1; j++) {
            // Compare adjacent elements and swap if out of order
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}
#endif
```

OUTPUT

```
Unsorted Array:
54 52 49 26 25 32 11 85 95 0 33 41 2 95 76 74 64 12 20 24 46 58 60 3 84
38 78 45 81 73 70 44 98 40 9 73 53 55 51 15 92 74 65 80 38 22 37 60 17 63
3 68 41 64 65 82 35 92 21 65 29 84 85 40 2 97 36 10 93 15 27 49 7 78 18
39 64 56 36 32 41 23 60 62 35 52 74 9 31 72 98 14 11 14 3 46 68 6 2 15

Bubble Sort:
0 2 2 2 3 3 3 6 7 9 9 10 11 11 12 14 14 15 15 15 17 18 20 21 22 23 24 25
26 27 29 31 32 32 33 35 35 36 36 37 38 38 39 40 40 41 41 41 44 45 46 46
49 49 51 52 52 53 54 55 56 58 60 60 60 62 63 64 64 64 65 65 65 68 68 70
72 73 73 74 74 74 76 78 78 80 81 82 84 84 85 85 92 92 93 95 95 97 98 98
```

OBSERVATION

The program compares neighboring elements in the array and swaps them if they are out of order. After applying bubbleSort(), the values are rearranged from smallest to largest.

Table 7-2. Bubble Sort Technique

CODE:Main

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>

//helper function
template <typename T>
int routine_smallest(T A[], int maxSize, int k){ //int k = current index;

    int position = k, j;
    T smallestElement = A[k];

    for (int J = k+1; J < maxSize; J++) {
        if (A[J] < smallestElement) {
            smallestElement = A[J];
            position = J;
        }
    }
    return position;
}

//selection sort
template <typename T>
void selectionSort(T arr[], int arrSize) {
    int POS, temp;

    for (int i = 0; i < arrSize; i++) {
        POS = routine_smallest(arr, arrSize, i);

        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;

    }
}
#endif
```

CODE:Header

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>

//helper function
template <typename T>
int routine_smallest(T A[], int maxSize, int k){ //int k = current index;

    int position = k, j;
    T smallestElement = A[k];

    for (int J = k+1; J < maxSize; J++) {
        if (A[J] < smallestElement) {
            smallestElement = A[J];
            position = J;
        }
    }
    return position;
}

//selection sort
template <typename T>
void selectionSort(T arr[], int arrSize) {
    int POS, temp;

    for (int i = 0; i < arrSize; i++) {
        POS = routine_smallest(arr, arrSize, i);

        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;

    }
}
#endif
```

OUTPUT

```
Unsorted Array:
40 81 85 6 6 96 77 38 25 97 58 41 53 90 4 2 99 20 7 23 95 12 52 85 69 19
92 12 71 31 25 14 23 23 79 14 83 16 77 66 36 1 99 70 26 7 87 97 68 18 35
30 72 36 64 75 2 75 73 93 50 69 81 13 67 94 51 29 45 10 63 45 97 15 37 24
88 21 54 62 81 68 96 56 67 86 63 42 82 2 65 63 13 60 85 42 76 78 44 16

Selection Sort:
1 2 2 2 4 6 6 7 7 10 12 12 13 13 14 14 15 16 16 18 19 20 21 23 23 23 24 25
25 26 29 30 31 35 36 36 37 38 40 41 42 42 44 45 45 50 51 52 53 54 56 58 60
62 63 63 63 64 65 66 67 67 68 68 69 69 70 71 72 73 75 75 76 77 77 78 79 81
81 81 82 83 85 85 85 86 87 88 90 92 93 94 95 96 96 97 97 97 99 99
```

OBSERVATION

The algorithm uses a helper function (routine_smallest) to repeatedly find the smallest element from the unsorted portion of the array. The located smallest element is swapped into its correct position, successfully sorting the 100 values in ascending order.

Table 7-3. Selection Sort Algorithm

CODE:Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingalgo.h"

//bubble sort
// generate 100 random values
// create a function that will display the random generated values

const int maxSize = 100;
void displayArr(int arr[]);

int main() {
    // create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    // use time(0) as seed for random values
    std::srand(std::time(0));

    // populate the array with random values
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = std::rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArr(unsortedArr);

    std::cout << "\nInsertion Sort:\n";
    insertionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    return 0;
}
void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

CODE:Header

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>

//insertion sort
template <typename T>
void insertionSort(T arr[], int arrSize) {
    int k = 1, j;
    T temp;

    while (k < arrSize) {
        temp = arr[k];
        j = k - 1;

        // Added 'j >= 0' to prevent out-of-bounds array access (arr[-1])
        while (j >= 0 && temp < arr[j]) {
            arr[j + 1] = arr[j];
            j--;   // responsible for swapping
        }
        arr[j + 1] = temp; // responsible for swapping
        k++; // responsible for swapping
    }
}
#endif
```

OUTPUT

```
Unsorted Array:
3 64 87 34 35 88 10 88 49 10 37 10 63 51 41 60 53 81 66 16 14 73 85 84 78 59 32 61 14 64 30 66 48
65 44 16 85 78 34 48 16 37 78 92 17 21 26 22 86 81 52 87 73 29 99 2 42 3 35 31 38 41 14 43 60 46 73
29 58 62 49 34 28 64 98 17 53 63 23 4 50 70 3 75 3 3 79 47 62 69 86 37 87 54 93 16 26 5 11 63

Insertion Sort:
2 3 3 3 3 3 4 5 10 10 10 11 14 14 14 16 16 16 16 17 17 21 22 23 26 26 28 29 29 30 31 32 34 34 34 35
35 37 37 37 38 41 41 42 43 44 46 47 48 48 49 49 50 51 52 53 53 54 58 59 60 60 61 62 62 63 63 63 64
64 64 65 66 66 69 70 73 73 73 75 78 78 78 79 81 81 84 85 85 86 86 87 87 87 88 88 92 93 98 99
```

OBSERVATION

The code takes one number at a time and slides it into its correct spot among the already sorted numbers. It includes a safety check (j >= 0) to stop the program from looking past the start of the array and crashing. The final output shows all 100 numbers rearranged from lowest to highest.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE:Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "suppselectionalgo.h"

const int maxSize = 100;
void displayArr(int arr[]);

int main()
{
    std::srand(std::time(0));
    int unsortedArr[maxSize];

    // random votes
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = (std::rand() % 5) + 1;
    }
    std::string candidates[5] = {
        "Bo Dalton Capistrano", "Cornelius Raymon Agustin", "Deja Jayla Banaga",
        "Lalla Brielle Yabut", "Franklin Relano Castro"
    };

    std::cout << "Unsorted Array:\n";
    displayArr(unsortedArr);

    // sorting votes using header file
    selectionSort(unsortedArr, maxSize);

    std::cout << "\nSorted Array:\n";
    displayArr(unsortedArr);

    // manual count
    int manualCount[6] = {0};
    for (int i = 0; i < maxSize; i++)
    {
        manualCount[unsortedArr[i]]++;
    }

    // algorithm count
    int voteCount[6] = {0};
    for (int i = 0; i < maxSize; i++)
    {
```

```
44          int vote = unsortedArr[i];
45          if (vote >= 1 && vote <= 5)
46          {
47              voteCount[vote]++;
48          }
49      }
50
51      // finding result/winnder
52      int winner = 1;
53      for (int i = 2; i <= 5; i++) {
54          if (voteCount[i] > voteCount[winner]) {
55              winner = i;
56          }
57      }
58
59      // display results
60      std::cout << "\nVote Count Results:";
61
62      for (int i = 1; i <= 5; i++) {
63          std::cout << "\nCandidate " << i << ": ";
64          std::cout << candidates[i - 1] << "\n";
65
66          std::cout << "Manual Count: ";
67          std::cout << manualCount[i] << "\n";
68
69          std::cout << "Algorithm Count: ";
70          std::cout << voteCount[i] << "\n";
71      }
72
73      std::cout << "\nWinning Candidate: Candidate " << winner << "\n";
74      std::cout << "Name: " << candidates[winner - 1] << "\n";
75      std::cout << "Votes: " << voteCount[winner] << "\n";
76
77      return 0;
78  }
79
80  void displayArr(int arr[]) {
81      for (int i = 0; i < maxSize; i++) {
82          std::cout << arr[i] << " ";
83      }
84      std::cout << std::endl;
85  }
```

CODE:Header

```
#ifndef SELECTIONALGO_H
#define SELECTIONALGO_H

template <typename T>
int routine_smallest(T A[], int maxSize, int k) {
    int position = k;
    T smallestElement = A[k];

    for (int j = k + 1; j < maxSize; j++) {
        if (A[j] < smallestElement) {
            smallestElement = A[j];
            position = j;
        }
    }
    return position;
}

// selection Sort
template <typename T>
void selectionSort(T arr[], int arrSize) {
    for (int i = 0; i < arrSize - 1; i++) {
        int pos = routine_smallest(
            arr, arrSize, i
        );

        T temp = arr[i];
        arr[i] = arr[pos];
        arr[pos] = temp;
    }
}
#endif
```

OUTPUT

```
Unsorted Array:
4 2 4 3 4 3 4 2 2 2 4 5 5 1 1 1 5 5 4 3 2 1 1 1 4 3 1 3 5 5 5 5 1 4 1 5 4 3 4 3 4 5 3 5 3 4 5 5 4 5
1 2 5 1 1 2 3 1 3 2 3 5 5 5 4 1 4 1 3 4 4 1 4 3 1 5 1 5 1 5 1 4 4 1 4 1 2 4 1 4 4 5 1 4 2 3 3 5 4 1

Sorted Array:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote Count Results:
Candidate 1: Bo Dalton Capistrano
Manual Count: 25
Algorithm Count: 25

Candidate 2: Cornelius Raymon Agustin
Manual Count: 10
Algorithm Count: 10

Candidate 3: Deja Jayla Banaga
Manual Count: 16
Algorithm Count: 16

Candidate 4: Lalla Brielle Yabut
Manual Count: 26
Algorithm Count: 26

Candidate 5: Franklin Relano Castro
Manual Count: 23
Algorithm Count: 23

Winning Candidate: Candidate 4
Name: Lalla Brielle Yabut
Votes: 26
```

PSEUDOCODE

```
START
    Generate 100 random votes (1-5) into unsortedArr

    Print unsortedArr
    Sort unsortedArr using Selection Sort
    Print sorted unsortedArr

    Count votes for each candidate (1-5)
    Find candidate with highest votes
    Print vote totals and winner
END

selectionSort(arr, size):
    For i = 0 to size - 1
```

```
    pos = routine_smallest(arr, size, i)
    Swap arr[i] and arr[pos]
  End For

routine_smallest(arr, size, start):
  Find index of lowest value from start to end of arr
  Return index
```

QUESTION
Question: Was your developed vote counting algorithm effective? Why or why not - Yes, the algorithm used was effective. Using selection sort, sorting the votes first grouped the same numbers together, making them easy to count and pick the winner without making mistakes.
CONCLUSION
I learned how bubble sort, selection sort, and insertion sort compare adjacent elements, find minimum values, or mmove elements to organize data in ascending order. Outputting both the unsorted and sorted arrays made it easy to make sure that each sorting algorithm ran properly. Applying Selection Sort to 100 random vote entries (values 1 through 5) successfully grouped identical votes together. Demonstrating how sorting algorithms simplify real-world data tasks like finding a winning candidate.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini.**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, grammar correction.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for Bug checking, and finding ways how to fix errors.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.